

Extrato Bancário Multiconta e Multimoeda: Uma Implementação com Microsserviços e Saga Orquestrado

Julli Mayanne Araújo¹

¹ Universidade Federal do Piauí (UFPI)
Teresina – PI – Brasil

jullimayanne9@gmail.com

Abstract. *This paper presents a functional demonstration of a multi-account, multi-currency bank statement system built on a polyglot microservices architecture. The system employs the Orchestrated Saga pattern for distributed transaction coordination, CQRS for read/write separation, Event Sourcing for immutable audit trails, the Transactional Outbox pattern for reliable event delivery, and double-entry bookkeeping for accounting integrity. Six microservices – implemented in Java, Go, Python, and Node.js – communicate asynchronously via Redpanda (Kafka-compatible broker), each with an independently chosen database (PostgreSQL, MongoDB, or Redis). A chaos testing suite validates resilience under concurrent load and service failures. The complete implementation is open-source and reproducible via Docker Compose¹.*

Resumo. *Este artigo apresenta uma demonstração funcional de um sistema de extrato bancário multiconta e multimoeda construído sobre uma arquitetura polígota de microsserviços. O sistema emprega o padrão Saga Orquestrado para coordenação de transações distribuídas, CQRS para separação de leitura e escrita, Event Sourcing para trilhas de auditoria imutáveis, o padrão Transactional Outbox para entrega confiável de eventos e contabilidade de dupla-entrada. Seis microsserviços – implementados em Java, Go, Python e Node.js – comunicam-se assincronamente via Redpanda (broker compatível com Kafka), cada um com banco de dados independente (PostgreSQL, MongoDB ou Redis). Uma suíte de testes de caos valida a resiliência sob carga concorrente e falhas de serviço. A implementação completa é open-source e reproduzível via Docker Compose.*

1. Introdução

Sistemas bancários modernos enfrentam o desafio de gerenciar múltiplas contas e moedas simultaneamente, mantendo consistência e rastreabilidade em operações distribuídas [Kleppmann 2017]. A arquitetura monolítica tradicional, embora funcional para cenários simples, apresenta limitações de escalabilidade e evolução independente quando a complexidade do domínio cresce.

Este trabalho apresenta uma implementação funcional de um sistema de extrato bancário multiconta e multimoeda utilizando uma arquitetura de microsserviços

¹Repository: <https://github.com/jjullimayanne/bank-statement-microservices>

políglota. A solução coordena transações distribuídas através do padrão **Saga Orquestrado** [Garcia-Molina and Salem 1987, Richardson 2018], garantindo consistência eventual sem o uso de transações distribuídas tradicionais (2PC).

A contribuição principal é a demonstração prática da integração de múltiplos padrões arquiteturais – CQRS, Event Sourcing, dupla-entrada contábil e banco-por-serviço – em um sistema que pode ser reproduzido com um único comando (`docker-compose up`). Além disso, cada microserviço utiliza a linguagem de programação mais adequada para sua responsabilidade, demonstrando a heterogeneidade que a arquitetura de microserviços viabiliza [Newman 2015].

O uso de ferramentas de IA generativa (Devin, da Cognition AI) foi empregado como auxílio na geração de código boilerplate e estruturação do artigo, sob revisão e validação integral dos autores humanos.

2. Arquitetura Proposta

O sistema é composto por seis microserviços, um message broker e um frontend de demonstração, conforme a Tabela 1.

Tabela 1. Microserviços e suas características

Serviço	Stack	Banco	Responsabilidade
Orchestrator	Java/Spring Boot	PostgreSQL	Coordenação da Saga
Account	Java/Spring Boot	PostgreSQL	Gestão de contas
Ledger	Java/Spring Boot	PostgreSQL	Dupla-entrada contábil
Transaction	Go/Gin	Stateless	Ingestão de eventos
Exchange	Python/FastAPI	MongoDB	Câmbio de moedas
Statement	Node.js/Express	MongoDB+Redis	Extrato (CQRS)

2.1. Padrão Saga Orquestrado

O Orchestrator Service atua como coordenador central das transações distribuídas. Ao receber uma solicitação de transação, ele cria uma instância de saga e coordena a execução sequencial dos passos: (1) validação da conta de origem, (2) conversão cambial (se necessário), (3) registro no ledger com dupla-entrada, e (4) atualização do modelo de leitura do extrato. Cada passo é executado por um microserviço independente que consome comandos de tópicos Kafka e publica respostas em tópicos de reply [Richardson 2018].

Em caso de falha em qualquer etapa, o orquestrador inicia automaticamente comandos de compensação nos serviços que já completaram seus passos, revertendo a transação de forma consistente.

2.2. CQRS e Event Sourcing

O sistema separa explicitamente os caminhos de escrita e leitura [Kleppmann 2017]. O **Ledger Service** funciona como um event store imutável (append-only), registrando cada transação como um par débito/crédito seguindo o princípio de dupla-entrada contábil. O **Statement Service** mantém um modelo de leitura otimizado em MongoDB, atualizado assincronamente a partir dos eventos confirmados pelo ledger, com cache Redis para consultas frequentes.

Esta separação permite que o modelo de leitura seja otimizado para consultas complexas (filtros por moeda, período, tipo de transação) sem impactar a performance do caminho de escrita.

2.3. Database per Service

Seguindo o princípio de banco-de-dados-por-serviço [Newman 2015], cada microserviço possui seu próprio armazenamento, escolhido conforme a necessidade de garantias ACID:

- **PostgreSQL (ACID):** Orchestrator, Account e Ledger – serviços onde consistência forte e precisão contábil são obrigatórias.
- **MongoDB (NoSQL):** Exchange e Statement – modelos de referência e leitura, otimizados para consultas rápidas sem necessidade de transações ACID.
- **Redis:** Cache do Statement Service para extratos frequentes.
- **Stateless:** Transaction Service apenas valida e publica eventos no broker.

2.4. Comunicação Assíncrona

Toda comunicação entre serviços ocorre via **Redpanda**, um broker compatível com o protocolo Apache Kafka. Foram definidos 9 tópicos Kafka para o fluxo completo da saga: `transaction-events`, `account-validation`, `account-validation-reply`, `exchange-rate-request`, `exchange-rate-reply`, `ledger-commands`, `ledger-confirmed`, `statement-updates` e `compensation-commands`.

O uso de Redpanda em vez de Apache Kafka traz vantagens de menor latência e simplicidade operacional, sem necessidade de ZooKeeper, mantendo total compatibilidade com clientes Kafka existentes.

3. Implementação

3.1. Stack Políglota

Uma das vantagens fundamentais da arquitetura de microserviços é a possibilidade de cada serviço utilizar a melhor linguagem para seu domínio [Newman 2015]. O **Transaction Service** foi implementado em Go (Gin framework) por sua alta performance em I/O concorrente e baixo consumo de memória – ideal para ingestão de eventos em alta vazão. O **Exchange Service** utiliza Python (FastAPI) pela simplicidade na integração com APIs externas de câmbio e MongoDB. O **Statement Service** foi construído em Node.js por seu modelo de I/O assíncrono e integração nativa com MongoDB e Redis. Os serviços críticos (Orchestrator, Account, Ledger) utilizam **Java com Spring Boot**, aproveitando o ecossistema maduro de JPA/Hibernate e Spring Kafka para garantias transacionais robustas.

3.2. Dupla-Entrada Contábil

O Ledger Service implementa o princípio fundamental da contabilidade de dupla-entrada: toda transação gera simultaneamente um débito em uma conta e um crédito em outra. Por exemplo, em uma transferência de R\$ 1.000 de João para Maria, o ledger registra: débito na conta de João e crédito na conta de Maria, ambos com o mesmo valor e vinculados ao mesmo `sagaId`. Este princípio garante que o balanço total do sistema sempre some zero, facilitando reconciliação e auditoria.

3.3. Idempotência

Cada evento carrega um `idempotencyKey` único. O Ledger Service verifica esta chave antes de processar, garantindo que eventos duplicados (cenário comum em sistemas distribuídos com retry automático) não resultem em registros duplicados no ledger.

3.4. Padrão Outbox

Para garantir entrega confiável de eventos mesmo em cenários de falha do message broker, os serviços Java implementam o **Transactional Outbox Pattern**. Em vez de publicar eventos diretamente no Kafka, cada serviço persiste o evento em uma tabela `outbox_events` no PostgreSQL **na mesma transação** que a operação de negócio. Um agendador (*scheduler*) executa periodicamente, lendo eventos pendentes da tabela e publicando-os no Kafka. Se o broker estiver indisponível, os eventos permanecem seguros no banco e são republicados quando a conexão for restabelecida, com controle de retries e máximo de tentativas configurável.

3.5. Infraestrutura como Código

Toda a infraestrutura é definida em um arquivo `docker-compose.yml` que orquestra: Redpanda (broker), PostgreSQL (3 bancos lógicos), MongoDB, Redis, os 6 microsserviços, o frontend, Prometheus e Grafana. O comando `docker-compose up --build` inicializa o sistema completo, incluindo a criação automática de tópicos Kafka e seed de dados de demonstração.

3.6. Observabilidade

O sistema integra **Prometheus** para coleta de métricas e **Grafana** com dashboards pré-provisionados. Os serviços Java expõem métricas via Spring Boot Actuator com Micrometer (Prometheus registry), enquanto os serviços Go, Python e Node.js expõem endpoints `/metrics` nativos. O dashboard inclui: taxa de requisições HTTP, latência p95, status das sagas, eventos do outbox, uso de memória JVM, conexões de banco (HikariCP) e métricas de negócio (transações publicadas, conversões cambiais, entradas de extrato processadas).

4. Testes de Caos e Validação

Para validar a resiliência da arquitetura, foi desenvolvida uma suíte de testes de caos em Python que simula cenários adversos reais:

- **Flood Test:** Envio de centenas de transações simultâneas para avaliar throughput e latência sob carga.
- **Transferências Concorrentes:** Múltiplas transferências simultâneas entre as mesmas contas para verificar consistência de saldo.
- **Multi-Currency Storm:** Câmbios cruzados concorrentes (BRL→USD→EUR→GBP) para estressar o Exchange Service.
- **Service Kill:** Interrupção de um container durante processamento de saga para verificar se a compensação funciona corretamente.
- **Reconciliação:** Comparação automática de saldos entre Account Service, Ledger e Statement Service após as baterias de teste.

A suíte utiliza requisições HTTP assíncronas (httpx) com workers configuráveis e apresenta métricas de throughput, latência média, percentil 95 e taxa de sucesso para cada cenário.

5. Demonstração

O frontend mobile-first, construído em React/Next.js, oferece uma interface simplificada para demonstração do sistema:

1. **Tela de Login:** Seleção entre duas contas mockadas (João – BRL/USD/EUR e Maria – BRL/EUR/GBP).
2. **Painel de Saldos:** Visualização dos saldos multimoeda da conta selecionada.
3. **Simular Transferência:** Formulário para publicar evento no Kafka, com opções de transferência, depósito, saque e câmbio.
4. **Extrato Multimoeda:** Visualização do extrato com filtro por moeda, mostrando créditos e débitos.
5. **Saga Status:** Acompanhamento em tempo real dos passos da saga orquestrada, com polling automático até conclusão.

O frontend atua como proxy reverso via Next.js rewrites, roteando requisições para os microsserviços correspondentes sem expor portas individuais ao usuário final.

6. Conclusão e Trabalhos Futuros

Este trabalho demonstrou a viabilidade prática de um sistema de extrato bancário multiconta e multimoeda utilizando uma arquitetura de microsserviços políglota com Saga Orquestrado. A implementação integra padrões consolidados – CQRS, Event Sourcing, Transactional Outbox, dupla-entrada contábil e database-per-service – em um sistema reprodutível e testável.

A suite de testes de caos validou que a arquitetura mantém consistência sob carga concorrente e que o mecanismo de compensação da saga funciona adequadamente em cenários de falha. A escolha de banco de dados por serviço (ACID vs NoSQL) demonstrou que nem todos os microsserviços necessitam de garantias transacionais fortes, e que a seleção adequada impacta positivamente a performance.

Como trabalhos futuros, identificamos: (i) implementação de autenticação e autorização (OAuth 2.0), (ii) observabilidade distribuída com OpenTelemetry e Jaeger, (iii) integração com APIs reais de câmbio, (iv) deployment em Kubernetes com service mesh (Istio), e (v) implementação de event replay completo a partir do ledger para reconstrução do estado do sistema.

Referências

- Garcia-Molina, H. and Salem, K. (1987). Sagas. In *Proceedings of the ACM SIGMOD International Conference on Management of Data*, pages 249–259. ACM.
- Kleppmann, M. (2017). *Designing Data-Intensive Applications*. O'Reilly Media.
- Newman, S. (2015). *Building Microservices: Designing Fine-Grained Systems*. O'Reilly Media.
- Richardson, C. (2018). *Microservices Patterns: With Examples in Java*. Manning Publications.